Кот в коде

Практикум «Python для начинающих»: задачи 01–30

Дмитрий Нефедов

Как устроено занятие

- Задачи идут по порядку, от **01** до **30**.
- Перед каждой задачей — всё новое, что для неё нужно: зачем, синтаксис, пример на *другом* сюжете, вопрос.
- Потом слайд **«Практика: задача NN»** — открываем Контест и решаем сами.
- Решить все 30 за три пары не обязательно. Остановились — продолжаем с первой нерешённой задачи.

Как пользоваться этим PDF

↑ Список задач внизу каждого слайда ведёт к оглавлению. Номер задачи в оглавлении — ссылка на её теорию. Вопросы на слайдах — чтобы проверить себя. Ответы обсуждаем вместе.

Как решать и отправлять

1. Каждая задача — **отдельный файл**: `task01.py`, `task02.py`, … Одну программу «на все задачи» не делаем.
2. В Контесте выбирайте компилятор **(make) Python**.
3. Консольная задача печатает **только ответ**. Никаких «Введите число»: `input("Введите…")` тоже печатает текст, и ответ не совпадёт.
4. Оконная задача (08, 10, 15, 17, 23, 27, 30): отправляется **весь файл** — создание окна, функции, `mainloop()`.
5. Каркас из примечаний можно брать, но TODO нужно дописать: пустой каркас тесты не пройдёт.
6. Сначала запустите программу у себя на примерах из условия, потом отправляйте.

Окно на вашем компьютере и окно в Контесте

У вас

Настоящее окно Tkinter: вы сами вводите текст, выбираете строки, нажимаете кнопки.
В задачах 27 и 30 программа ходит в настоящий интернет — нужен пакет `requests`.

В Контесте

Окна на экране нет. Проверка сама вводит, выбирает и нажимает, а после каждого действия сверяет надписи, поля, окна сообщений и запросы.
Вместо интернета отвечает подготовленный тест.

Размещайте элементы через `pack` или `grid` (`place` проверка не поддерживает). Тексты надписей и кнопок — ровно как в условии.

Если ответ не принят

1. Откройте **сообщение чекера** у первого неудачного теста.
2. Консоль: там ввод, ожидаемый и ваш ответ. Окно: тест, шаг, что ввели или нажали, чем окно отличается.
3. Повторите эту ситуацию у себя и найдите правило, которое нарушено.
4. Ошибка Python? Сообщение называет её тип и строку программы.
5. Не помогло — подсказка 1, потом подсказка 2 в примечаниях, потом вопрос преподавателю.

Пробелы, регистр, знаки

«Кабинет 12. Сотрудник: Барсик» и «Кабинет 12 . Сотрудник: Барсик» — разные ответы.

Список задач

Строки и проверки

- 01 Табличка на двери
- 02 Сколько стоит доставка
- 03 Карточка питомца
- 04 Похоже ли на время
- 05 Бывает ли такое время
- 06 Таймер корма
- 07 Три исхода
- 08 Таймер корма в окне
- 09 Будильник для совы
- 10 Напоминалка в окне

Списки, словари, JSON

- 11 Фото или маскировка
- 12 Что выбрано в списке
- 13 Поиск по породам
- 14 Номер после поиска
- 15 Поиск породы
- 16 Справочник подписей
- 17 Подпись по выбору
- 18 Первый JSON
- 19 Курс внутри ответа
- 20 Ответ в несколько строк

JSON, API и сеть

- 21 Поле забыли прислать
- 22 Битый JSON
- 23 JSON-просмотрщик
- 24 Сервер ответил
- 25 Курс из ответа
- 26 Адрес запроса породы
- 27 Случайная собака
- 28 Код ответа
- 29 Четыре проверки по очереди
- 30 Валюты

Блок 1

Требование становится правилом

Задачи 01–03: точный вывод, границы, порядок проверок

01 Робот сравнивает символ в символ

Ответ в Контесте проверяет программа. Для неё лишний пробел или другая буква — уже другой ответ.

- ✔ `Кабинет 12. Сотрудник: Барсик`
- ✖ `Кабинет 12 . Сотрудник: Барсик`
- ✖ `кабинет 12. Сотрудник: Барсик`

Вспомним

`input()` читает одну строку и **всегда** возвращает строку. `print(a, b)` печатает значения через пробел и переводит строку.

01 Новое: f-строка

Пример: бейдж участника конференции. Ввод — две строки: `Анна Петрова` и Казань.

```
name = input()  # Анна Петрова
city = input()  # Казань
print("Участник", name, ".")  # print ставит пробел между частями
print(f"Участник {name}.")  # в f-строке пробелы ставите вы
print(f"Бейдж: {name} ({city})")
```

Вывод

```
Участник Анна Петрова .
Участник Анна Петрова.
Бейдж: Анна Петрова (Казань)
```

- `f` перед кавычкой: в `{}` — переменная или выражение.
- Всё остальное печатается как написано — каждый пробел и знак.
- `print` с запятыми сам ставит пробелы между частями.

01 Проверьте себя

Вопрос 01.1

Что напечатают две строки?

```
print("Ряд", 5, "!")
print(f"Ряд {5}!")
```

Вопрос 01.2

Ученик написал `name = input("Имя: ")`. Программа работает, но Контест не принимает ответ. Почему?

Практика: задача 01 Табличка на двери

Тренируем: точный вывод по образцу; f-строка

Откройте задачу **№01** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

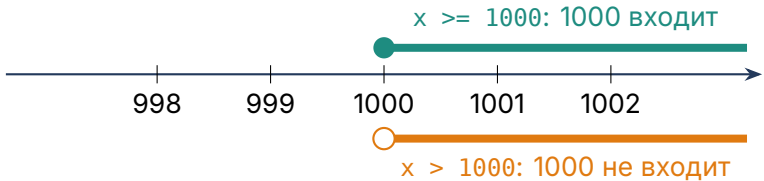
Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



02 «Большой заказ» — это сколько?

Заказчик сказал «дешевле для больших заказов». Проверить это нельзя, пока нет числа. Точное правило: **«от 1000»**.



«от 1000», «1000 и больше»	`x >= 1000`
«больше 1000»	`x > 1000`
«до 4999 включительно»	`x <= 4999`

02 Граница входит или нет

```
print(3 >= 3, 3 > 3)  # граница входит только в >=

for cups in [2, 3, 4]:
    if cups >= 3:
        print(cups, "— скидка")
    else:
        print(cups, "— без скидки")
```

Вывод

```
True False
2 — без скидки
3 — скидка
4 — скидка
```

Кофейня: скидка **от 3 чашек**. На границах ошибаются чаще всего: проверяйте число *перед* границей, саму границу и число после.

02 Несколько порогов: порядок важен

```
t = 35
# Плохо: широкое условие стоит первым
if t > 20:
    print("тепло")
elif t > 30:
    print("жарко")  # не выполнится никогда
# Хорошо: сначала самый высокий порог
if t > 30:
    print("жарко")
elif t > 20:
    print("тепло")
```

Вывод

```
тепло
жарко
```

В цепочке **`if` / `elif`** срабатывает **первое** истинное условие: широкое условие первым «съедает» узкое. Начинайте с самого строгого порога или пишите обе границы.

02 Проверьте себя

Вопрос 02.1

В «плохом» варианте со слайда выше `t = 25`. Что напечатается? А при `t = 20`?

Вопрос 02.2

Правило: «от 1000 до 4999 включительно». На каких трёх-четырёх числах стоит проверить программу?

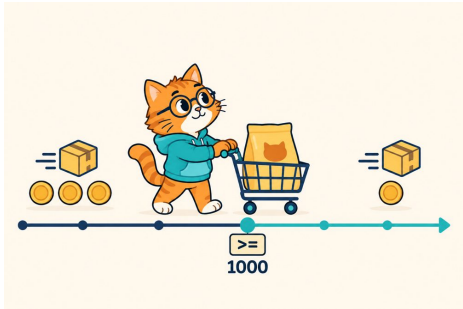
Практика: задача 02 Сколько стоит доставка

Тренируем: границы диапазонов: >= или >

Откройте задачу **№02** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



03 Пробелы по краям и длина строки

```
raw = "  Капитан Флинт  "
name = raw.strip()  # новая строка, raw не изменился
print(len(raw), len(name))  # пробел внутри тоже символ
print(f"[{raw}]")
print(f"[{name}]")
```

Вывод

```
17 13
[  Капитан Флинт  ]
[Капитан Флинт]
```

`s.strip()` возвращает **новую** строку без пробелов по краям; исходная не меняется — сохраните результат. Пробелы внутри остаются.
`len(s)` — число символов, пробелы тоже считаются.

03 Перебор символов строки

Сколько цифр в номере автомобиля?

```
plate = "A123BC"
count = 0
for ch in plate:  # ch — по очереди каждый символ
    if ch in "0123456789":
        count = count + 1
print("цифр:", count)
```

Вывод

```
цифр: 3
```

`for ch in plate:` — ch по очереди принимает каждый символ строки. `ch in "0123456789"` — есть ли этот символ среди цифр. Так проверяют «это цифра?».

03 Сообщаем первую нарушенную проверку

```
n = 150
if n > 100:
    print("больше 100")
elif n % 2 == 0:
    print("чётное")  # тоже верно, но не напечатается
else:
    print("ни то ни другое")
```

Вывод

```
больше 100
```

150 и больше 100, и чётное. Напечатано только первое: **`elif`** проверяется, лишь когда всё выше не сработало.

Если условие перечисляет правила по порядку, этот порядок — тоже требование. Сообщаем **первое** нарушенное правило.

03 Проверьте себя

Вопрос 03.1

Что напечатает программа со слайда выше при `n = 8`? А при `n = 7`?

Вопрос 03.2

```
s = "  Ми  "
s.strip()
print(len(s))
```

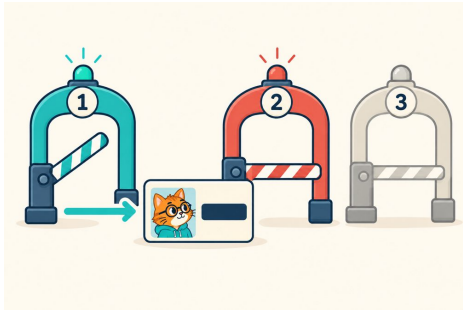
Практика: задача 03 Карточка питомца

Тренируем: `strip`, `len`, проверка цифр, порядок правил

Откройте задачу **№03** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.

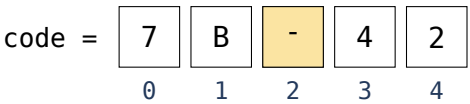


Блок 2

Время ЧЧ:ММ

Задачи 04–05: форма строки и смысл значения

04 Номер символа в строке



номера с **нуля**: у 5 символов последний — номер 4

```
code = "7B-42"
print(len(code))
print(code[0], code[2])
for i in [0, 3, 4]:
    digit = code[i] in "0123456789"
    print(i, code[i], digit)
```

Вывод

```
5
7 -
0 7 True
3 4 True
4 2 True
```

`code[i]` — символ с номером `i`. `len(code)` — сколько всего символов.

04 Сначала длина, потом позиции

```
code = "7B"
print(len(code))
print(code[4])  # такого символа нет
```

Вывод

```
2
IndexError: string index out of ra
```

Символа с таким номером нет — программа падает с `IndexError`.

Поэтому порядок: сначала `len(s)`, и только если длина подходит, — отдельные позиции.

Пробел — тоже символ. Строку не «чистите», если условие этого не просит.

04 Проверьте себя

Вопрос 04.1

`s = "12:3"`. Что будет при `print(s[4])`? Чему равно `len(s)`?

Вопрос 04.2

Строка `" 9:30"`. Сколько в ней символов и какой символ стоит под номером 0?

Практика: задача 04 Похоже ли на время

Тренируем: длина строки и проверка символов по позициям

Откройте задачу **№04** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



05 Срез строки

```
s = "ПН-2024-07"
print(s[0:2])        # номера 0, 1
print(s[3:7])        # номера 3, 4, 5, 6
print(int(s[8:10]))  # "07" → 7
level = 12
print(0 <= level <= 10)
print(level >= 0 and level <= 10)
```

Вывод

```
ПН
2024
7
False
False
```

`s[a:b]` — символы с номера `a` до `b`, **не включая** `b`. Получается строка.

`int("07")` даёт 7: ведущий ноль `int` понимает.

05 Форма и смысл — две разные проверки

Строка	Форма `ЧЧ:ММ`	Такое время бывает	Почему
`12:30`	✔	✔	
`24:00`	✔	✖	часов только 0–23
`12:60`	✔	✖	минут только 0–59
`9:05`	✖	—	не 5 символов

Сначала форма (задача 04), потом смысл: срезом достаём часы и минуты, `int` превращает их в числа, и числа сравниваем с границами.

Двойное сравнение: `0 <= x <= 23` — то же, что `x >= 0 and x <= 23`.

05 Проверьте себя

Вопрос 05.1

`s = "23:59"`. Что вернут `s[0:2]`, `s[3:5]` и `int(s[3:5]) + 1`?

Вопрос 05.2

В примере со слайда «Срез строки» заменили `level = 12` на `level = 7`. Что напечатают две последние строки?

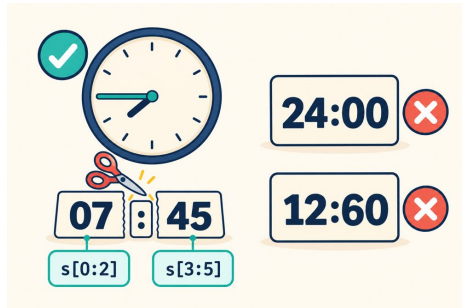
Практика: задача 05 Бывает ли такое время

Тренируем: срезы, `int`, границы часов и минут

Откройте задачу **№05** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



Блок 3

Ошибки ввода: try / except

Задачи 06–08: программа не падает на неверном вводе

06 Строка и число — не одно и то же

```
text = input()  # "5" — строка
print(text + text)  # 55
n = int(text)  # 5 — число
print(n + n)  # 10
```

Фрагмент: ввод — строка 5.

`int()` превращает строку в целое число. Если не получается, Python останавливает программу:

Ошибка

```
ValueError: invalid literal for
int() with base 10: 'пять'
```

Пользователь может ввести что угодно, а программа не должна падать.

06 try / except: куда идёт программа

```
for text in ["42", "сорок"]:
    print("1. начинаем:", text)
    try:
        n = int(text)
        print("2. получилось:", n)
    except ValueError:
        print("3. ошибка ValueError")
    print("4. идём дальше")
```

Вывод

```
1. начинаем: 42
2. получилось: 42
4. идём дальше
1. начинаем: сорок
3. ошибка ValueError
4. идём дальше
```

В **try** — то, что может упасть. Если там случилась `ValueError`, остаток **try** пропускается и выполняется **except**. Потом программа идёт дальше, как обычно.

06 Что считается числом для int()

```
samples = [" 15 ", "+7", "007", "-3",
           "1.5", "", "пять"]
for text in samples:
    try:
        n = int(text)
        print(f"[{text}] → {n}")
    except ValueError:
        print(f"[{text}] → не число")
```

Вывод

```
[ 15 ] → 15
[+7] → 7
[007] → 7
[-3] → -3
[1.5] → не число
[] → не число
[пять] → не число
```

Пробелы по краям, знак и ведущие нули `int()` принимает. Дробь и пустую строку — нет.

`"-3".isdigit()` — `False`, хотя `int("-3")` работает. Поэтому «число ли это» надёжнее проверять через **`try`**.

06 Проверьте себя

Вопрос 06.1

Что напечатает программа со слайда «try / except: куда идёт программа», если в списке будет строка `"4.0"`?

Вопрос 06.2

Почему в `try` лучше класть только опасную строку, а не всю программу?

Практика: задача 06 Таймер корма

Тренируем: `try / except ValueError`: неверный ввод не роняет программу

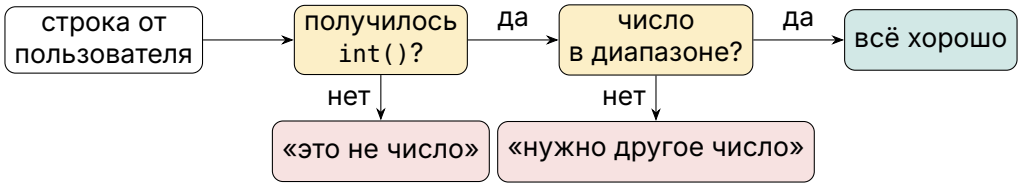
Откройте задачу **№06** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



07 Две разные ошибки — два разных сообщения



Неправильный формат: строку нельзя превратить в число (`abc`, `1.5`).
Значение вне диапазона: число получилось, но не подходит (`500`).
Пользователю нужно разное: в первом случае написать число, во втором — выбрать другое.

07 try / except / else

```
for text in ["8", "восемь"]:
    try:
        n = int(text)
    except ValueError:
        print(text, "→ except")
    else:
        print(text, "→ else, n =", n)  # если не было ошибки
```

Вывод

```
8 → else, n = 8
восемь → except
```

Блок **else** после **except** выполняется, **только если в try не было ошибки**. Значит, в нём число уже точно есть — там удобно проверять диапазон обычным **if**.

07 Проверьте себя

Вопрос 07.1

Диапазон «от 1 до 120 включительно». Для каждого ввода скажите, какой это случай: `abc`, `0`, `" 120 "`, `121`, `1.5`.

Вопрос 07.2

В примере `try` / `except` / `else` поменяли `"восемь"` на `""`. Какая ветка сработает?

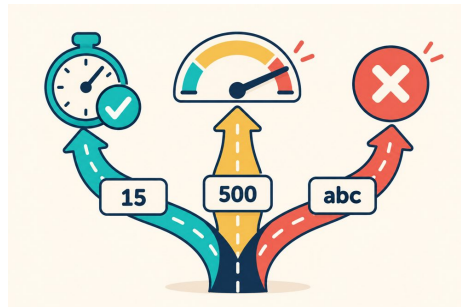
Практика: задача 07 Три исхода

Тренируем: не число, число вне диапазона, нормальное число

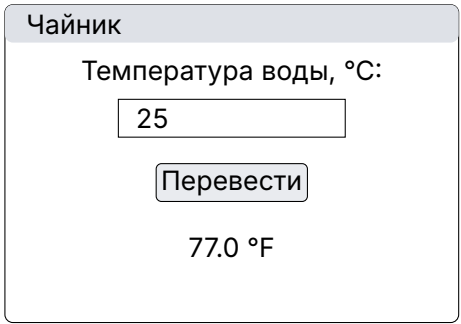
Откройте задачу **№07** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



08 Вспомним окно: виджеты и обработчик



- `Label` — надпись, `Entry` — поле ввода, `Button` — кнопка. `pack()` ставит их в окно сверху вниз.
- У кнопки `command=имя_функции`. Нажали — Tkinter вызывает эту функцию: это **обработчик**.
- Обработчик читает поле, считает и меняет надпись.
- `window.mainloop()` в конце держит окно открытым и ждёт нажатий.

Пример — перевод температуры воды из °C в °F. Полный файл: `demos/g08_tea.py`.

08 Окно примера: создаём виджеты

```
window = Tk()
window.title("Чайник")
window.geometry("300x160")

Label(text="Температура воды, °C:").pack()
entry = Entry()
entry.pack()
Button(text="Перевести", command=convert).pack()  # без скобок!
result = Label(text="Пока пусто")
result.pack()

window.mainloop()
```

Надпись результата создаётся **один раз**. Потом обработчик меняет только её текст — новых `Label` при каждом нажатии не делаем.

08 command=handler или command=handler()

```
# Ошибка: скобки вызывают функцию сразу, при создании кнопки
Button(text="Перевести", command=convert())
# Правильно: передаём саму функцию, её вызовет нажатие
Button(text="Перевести", command=convert)
```

Со скобками функция вызывается сразу — ещё при создании кнопки, до показа окна. В `command` попадает то, что она вернула (`None`), и потом нажатие ничего не делает.

Без скобок мы отдаём кнопке саму функцию. Вызовет её Tkinter — в момент нажатия.

Фрагмент: две строки показывают разницу, в программе нужна только вторая.

08 Обработчик: читаем поле, меняем надпись

```
def convert():
    text = entry.get()  # всегда строка, даже "25"
    try:
        celsius = float(text)
    except ValueError:
        result.config(text="Введите число")
        return  # дальше в функции не идём
    fahrenheit = celsius * 9 / 5 + 32
    result.config(text=f"{fahrenheit} °F")
```

- `entry.get()` — текст поля. Это **всегда строка**, даже если там цифры.
- `result.config(text=...)` — новый текст надписи.
- `return` — выйти из функции: после ошибки дальше считать нечего.

08 Переменная и текст надписи — разные вещи

```
status = "Пока пусто"
result = Label(text=status)

status = "Готово"  # окно не изменится: Label уже создан
result.config(text="Готово")  # окно изменится
```

Фрагмент. `Label` запомнил текст в момент создания. Новое значение переменной окно не видит: надпись меняется только через `config`.

08 Ошибка в обработчике и что видно в окне

Если обработчик упал, окно **не закрывается**. Сообщение об ошибке уходит в консоль, которую пользователь не видит, — в окне просто ничего не меняется.

Поэтому `try / except` стоит **внутри обработчика**, и на неверный ввод окно отвечает надписью.

Как меняется окно примера (проверено эмулятором):

Действие	Надпись
запуск	Пока пусто
ввели «25»	Пока пусто
нажали «Перевести»	77.0 °F
ввели «тепло»	77.0 °F
нажали «Перевести»	Введите число

08 Проверьте себя

Вопрос 08.1

В окне примера поле пустое, нажали «Перевести». Что покажет надпись?

Вопрос 08.2

В программе примера написали `command=convert()`. Что произойдёт при запуске? Подсказка: посмотрите, в каком порядке создаются кнопка и надпись `result`.

Вопрос 08.3

Ввели «25», нажали; потом стёрли поле и *не* нажали кнопку. Какая надпись?

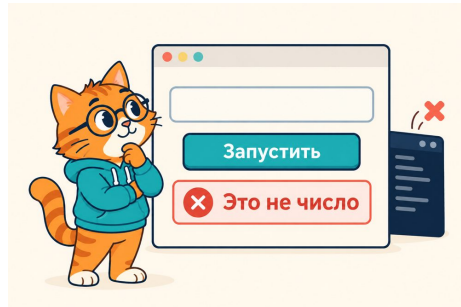
Практика: задача 08 Таймер корма в окне

Тренируем: `try/except` внутри обработчика, `entry.get()`, `config`. Настоящий отсчёт времени не нужен

Откройте задачу **№08** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: весь файл окна — создание окна, обработчики, `mainloop()`. Каркас — в примечаниях к задаче.

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



Блок 4

Напоминалка

Задачи 09–10: время числом и окно ошибки

09 Время одним числом

Сравнивать и вычитать «07:45» и «23:10» неудобно. Удобно превратить время в **минуты от полуночи**:

`07:45` → `7 * 60 + 45 = 465`

В сутках `24 * 60 = 1440` минут: от `0` (00:00) до `1439` (23:59).

Обратно — целым делением и остатком:

`465 // 60 = 7` (часы) `465 % 60 = 45` (минуты)

Часы и минуты из строки достаём срезами, как в задаче 05: `s[0:2]` и `s[3:5]`.

09 Остаток «заворачивает» по кругу

Дни недели: пн = 0, ..., вс = 6. После воскресенья снова понедельник.

```
print(125 // 60, 125 % 60)  # 2 ч 5 мин
today = 4                   # пятница
print((today + 4) % 7)      # 1 — вторник
print((1 - today) % 7)      # до вторника 4 д
print(1 - today)            # без % было бы
```

Вывод

```
2 5
1
4
-3
```

`%` 7 всегда даёт число от 0 до 6, даже для отрицательной разницы: «прошлое» на этой неделе превращается в «через столько дней».

Сутки — такой же круг, только из 1440 минут.

09 Проверьте себя

Вопрос 09.1

Чему равны `1439 // 60` и `1439 % 60`? Какое это время?

Вопрос 09.2

Сегодня среда (2). Через сколько дней ближайший понедельник (0)? Запишите выражение с `%`.

Вопрос 09.3

Что даст `(10 - 10) % 1440`?

Практика: задача 09 Будильник для совы

Тренируем: минуты от полуночи, `//` и `%`, переход через полночь

Откройте задачу **№09** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

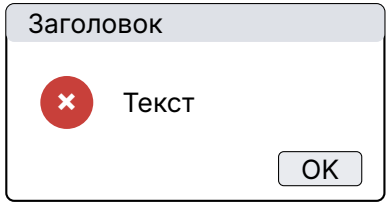
Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



10 Окно с ошибкой: messagebox.showerror

```
from tkinter import *
from tkinter import messagebox  # отдельно!

messagebox.showerror("Заголовок", "Текст")
```



- `from tkinter import *` модуль `messagebox` **не** подключает. Без второй строки — `NameError` при первом же вызове.
- Окно ошибки появляется, программа ждёт «ОК» и продолжает обработчик.

10 Проверку удобно вынести в функцию

```
def is_code(s):
    # True, если в строке ровно 6 цифр
    if len(s) != 6:
        return False
    for ch in s:
        if ch not in "0123456789":
            return False
    return True
```

Пример: окно «Приглашение» принимает код из 6 цифр.

Функция возвращает `True` или `False`. **`return`** сразу заканчивает функцию — дальше проверять не нужно.

Такую функцию можно вызвать для каждого поля окна. «Хотя бы одно неверно»:
`not` `ok1` **`or not`** `ok2`.

10 Обработчик: ошибка → окно и сброс надписи

```
def enter():
    if not is_code(entry.get()):
        messagebox.showerror("Ошибка", "Нужен код из 6 цифр")
        result.config(text="Вход не выполнен")  # сбросить успех
        return
    result.config(text="Добро пожаловать!")
```

Действие	Что в окне
123456, «Войти»	Добро пожаловать!
12a456, «Войти»	окно ошибки; надпись «Вход не выполнен»

Без сброса после ошибки в окне осталось бы старое «Добро пожаловать!» — неправда. Полный файл: `demos/g10_invite.py`.

10 Проверьте себя

Вопрос 10.1

Забыли строку `from tkinter import messagebox`. Окно открывается. Что случится при вводе неверного кода и нажатии «Войти»?

Вопрос 10.2

Пользователь трижды нажал «Войти» с неверным кодом. Сколько окон ошибки появится?

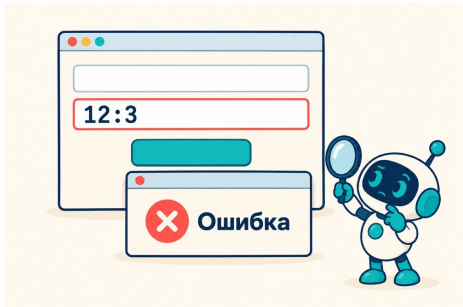
Практика: задача 10 Напоминалка в окне

Тренируем: `messagebox.showerror`, проверка двух полей, сброс надписи после ошибки. Звонок будильника не нужен

Откройте задачу **№10** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: весь файл окна — создание окна, обработчики, `mainloop()`. Каркас — в примечаниях к задаче.

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



Блок 5

Фотоархив

Задача 11: имя файла и его окончание

11 Регистр и окончание строки

```
print("Отчёт.PDF".lower())  # отчёт.pdf
print("+7 900 123".startswith("+7"))  # True
print("отчёт.pdf.exe".endswith(".pdf"))  # False
print("pdf" in "отчёт.pdf.exe")  # True — in ищет везде
print("Урок.MP4".lower().endswith((".mp4", ".avi")))  # True
```

Вывод

```
отчёт.pdf
True
False
True
True
```

- `s.lower()` — копия строки маленькими буквами.
- `s.endswith(x)` — кончается ли строка на `x`. Можно передать несколько вариантов в круглых скобках.
- `x in s` ищет *где угодно* — для окончаний не годится.

11 Проверьте себя

Вопрос 11.1

Что вернут выражения?

```
"Отпуск.Jpeg".lower().endswith((".jpg", ".png"))
"photojpg".endswith("jpg")
"photojpg".endswith(".jpg")
```

Вопрос 11.2

Зачем вызывать `lower()` до `endswith`, а не после?

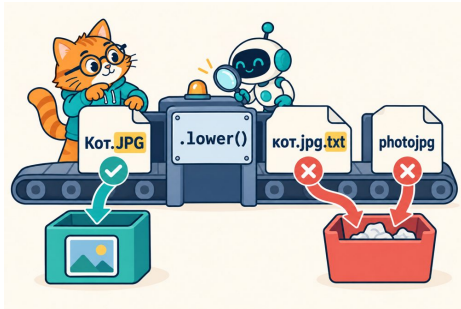
Практика: задача 11 Фото или маскировка

Тренируем: `lower()` и `endswith()` с несколькими окончаниями

Откройте задачу **№11** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



Блок 6

Список Listbox

Задачи 12–15: выбор в списке, поиск и видимый список

12 Вспомним Listbox и curselection()



`curselection()` → `(2,)`

- Строки списка нумеруются с нуля, как символы строки.
- `listbox.curselection()` возвращает **кортеж** номеров выбранных строк: `(2,)` — выбрана третья.
- Ничего не выбрано — пустой кортеж `()`. В `if` он считается ложью.
- Номер — это `selection[0]`. Сам текст строки берут по номеру из списка.

В задаче 12 окна ещё нет: три строки из условия изображают список и выбор, их нужно скопировать.

12 Кортеж номеров и пустой выбор

```
songs = ["Утро", "Дорога", "Вечер"]
selection = (2,)  # выбрана третья строка
print(selection[0])
print(songs[selection[0]])
empty = ()
print(len(empty), bool(empty))
if not empty:
    print("ничего не выбрано")
```

Вывод

```
2
Вечер
0 False
ничего не выбрано
```

`bool(())` — `False`, поэтому **`if not`** `selection:` читается как «если ничего не выбрано». Обращаться к `selection[0]` можно только после этой проверки.

12 Проверьте себя

Вопрос 12.1

`songs = ["Утро", "Дорога", "Вечер"]` и `selection = (0,)`. Что такое `songs[selection[0]]`?

Вопрос 12.2

Что случится, если сначала взять `selection[0]`, а пустоту проверять потом?

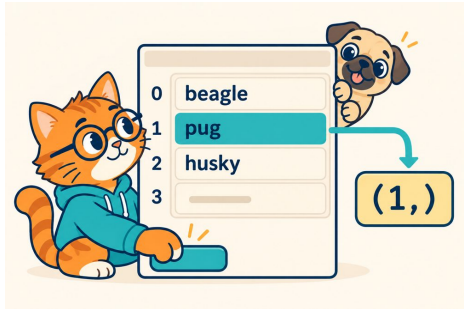
Практика: задача 12 Что выбрано в списке

Тренируем: кортеж из `curselection()`, пустой выбор

Откройте задачу **№12** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



13 Поиск кусочка в строке

```
films = ["Лесной дух", "Город",
         "Тайга", "Лес и море"]
long_titles = []  # новый список
for film in films:
    if len(film) > 5:
        long_titles.append(film)
print(long_titles)
print(len(long_titles))
print("лес" in "Лесной дух")  # регистр
print("лес" in "Лесной дух".lower())
print("" in "Город")  # пусто: есть везде
```

Вывод

```
['Лесной дух', 'Лес и море']
2
False
True
True
```

`часть in строка` — есть ли кусочек внутри; регистр важен, поэтому `lower()`. Найденное — в **новый** список через append. Строку «a, b» в список: `"a, b".split(", ")`.

13 Проверьте себя

Вопрос 13.1

Что вернут `"ГОР" in "Город"`, `"гор" in "Город".lower()` и `"  ".strip() in "Тайга"`?

Вопрос 13.2

Чем `"море" in "Лес и море"` отличается от `"Лес и море".startswith("море")`?

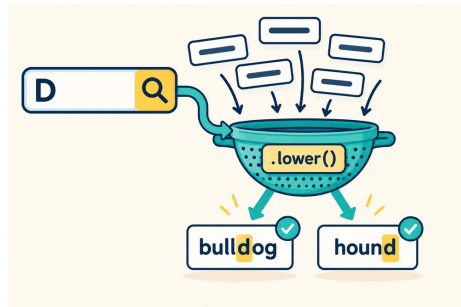
Практика: задача 13 Поиск по породам

Тренируем: поиск подстроки без учёта регистра, новый список найденного

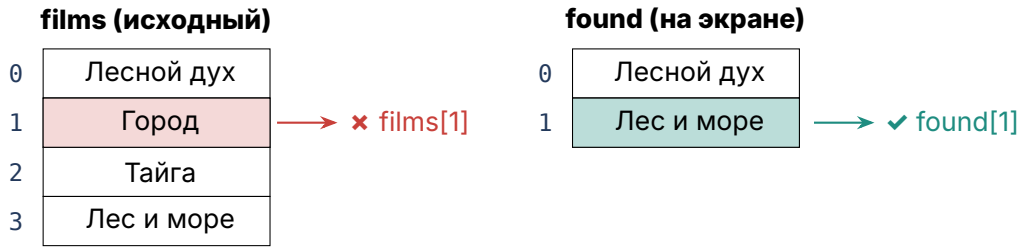
Откройте задачу **№13** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



14 Исходный список и список после поиска



После поиска на экране — другой, короткий список. Человек выбрал вторую строку *того, что видит*. Номер 1 относится к найденному списку, а в исходном под этим номером совсем другой элемент.

14 Пример и проверка

```
films = ["Лесной дух", "Город", "Тайга", "Лес и море"]
found = ["Лесной дух", "Лес и море"]  # после поиска «лес»
choice = 1  # выбрана вторая строка на экране
print(found[choice])  # Лес и море — то, что видит человек
print(films[choice])  # Город — чужая строка
```

Элемент берут из того списка, который сейчас на экране.

Вывод

```
Лес и море
Город
```

Вопрос 14.1

Найдено `["Тайга"]`, выбрана строка 0. Что вернут `found[0]` и `films[0]`?

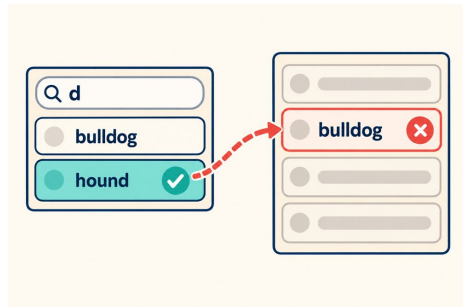
Практика: задача 14 Номер после поиска

Тренируем: номер выбора относится к найденному списку

Откройте задачу **№14** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



15 Методы Listbox

`lb.insert(END, "Плов")`	добавить строку в конец
`lb.get(1)`	текст строки с номером 1 (вторая сверху)
`lb.delete(0, END)`	удалить все строки; выделение тоже пропадает
`lb.curselection()`	кортеж номеров выбранных строк: `(1,)` или `()`

`END` — «конец списка», приходит из `from tkinter import *`. `lb` — имя вашего `Listbox`.

Пример — меню кафе: кнопки «Завтраки» и «Обеды» заполняют список заново, «Убрать из меню» удаляет выбранную строку. Полный файл: `demos/g15_menu.py`.

15 Заполнить список заново

```
def fill(dishes):
    menu_list.delete(0, END)  # убрать все строки (и выделение)
    for dish in dishes:
        menu_list.insert(END, dish)  # добавить в конец


def show_breakfast():
    fill(BREAKFAST)
```

Сначала `delete(0, END)`, потом `insert` для каждого элемента. Без `delete` новые строки добавятся к старым.

Кнопке нужна функция без аргументов, поэтому у «Завтраков» и «Обедов» свои маленькие функции, которые вызывают `fill` с нужным списком.

15 Выбранная строка — из видимого списка

```
def remove():
    selection = menu_list.curselection()  # (номер,) или ()
    if selection:
        dish = menu_list.get(selection[0])  # из видимого списка
        menu_list.delete(selection[0])  # удалить одну строку
        result.config(text=f"Убрали: {dish}")
```

Выбор, потом «Убрать»	Надпись
ничего не выбрано	без изменений
2-я строка сверху	Убрали: Сырники
«Обеды», 1-я строка	Убрали: Борщ

`menu_list.get(номер)` берёт текст прямо из списка на экране. `delete(номер)` удаляет одну строку.

15 Проверьте себя

Вопрос 15.1

Убрали `menu_list.delete(0, END)` из `fill`. Что будет в списке после нажатия «Обеды»?

Вопрос 15.2

Выбрали «Омлет», потом нажали «Обеды», потом «Убрать из меню». Что произойдёт и почему?

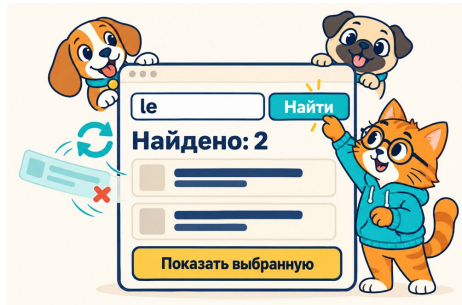
Практика: задача 15 Поиск породы

Тренируем: перезаполнение Listbox, выбор из видимого списка

Откройте задачу **№15** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: весь файл окна — создание окна, обработчики, `mainloop()`. Каркас — в примечаниях к задаче.

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



Блок 7

Словарь и события

Задачи 16–17: словарь и событие выбора

16 Словарь: ключ → значение

```
floors = {"бухгалтерия": 2, "склад": 0, "кухня": 1}
print(floors["склад"])  # 0
print("архив" in floors)  # False
print(len(floors))  # 3 пары
floors["архив"] = -1  # добавили пару
print(floors)
```

Вывод

```
0
False
3
{'бухгалтерия': 2, 'склад': 0, 'кухня': 1, 'архив': -1}
```

Значение — по ключу в квадратных скобках; ключ `in` словарь проверяет ключи. Присваивание по новому ключу добавляет пару.

16 Ключа нет — KeyError

```
floors = {"склад": 0}
print(floors["архив"])
```

Вывод

```
KeyError: 'архив'
```

Обращение по отсутствующему ключу роняет программу. Поэтому сначала спрашивают `"архив" in floors` и только при `True` берут значение.

Ключи сравниваются точно: `"Склад"` и `"склад"` — разные ключи.

Вопрос 16.1

Что вернут `2 in floors` и `"кухня" in floors` для словаря с прошлого слайда?

Практика: задача 16 Справочник подписей

Тренируем: значение по ключу, проверка ключа через `in`

Откройте задачу **№16** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



17 Реакция на выбор без кнопки: bind

Кнопка

```
def go():
    ...
Button(text="OK", command=go)
```

Функция без аргументов. Вызывается при нажатии.

Выбор в списке

```
def on_select(event):
    ...
lb.bind("<<ListboxSelect>>", on_select)
```

Функция получает **один аргумент** — объект события. Вызывается, когда выбор строки меняется: выбрали или *сняли*.

Фрагменты. В обоих случаях функцию передают **без скобок**. Параметр `event` нужен, даже если вы им не пользуетесь: иначе `TypeError: on_select() takes 0 positional arguments but 1 was given`.

17 Пример: город по выбору

```
def on_select(event):  # bind передаёт объект события
    selection = city_list.curselection()
    if not selection:  # выбор сняли
        info.config(text="Ничего не выбрано")
        return
    city = city_list.get(selection[0])
    info.config(text=f"Город: {city}")
```

```
city_list.bind("<<ListboxSelect>>", on_select)  # без скобок
```

Действие (эмулятор окна)	Надпись
запуск	Ничего не выбрано
выбрали «Сочи»	Город: Сочи
сняли выбор	Ничего не выбрано

Полный файл: `demos/g17_cities.py`.

17 Проверьте себя

Вопрос 17.1

Ученик написал `def on_select():` без `event`. Окно открывается. Что произойдёт при выборе строки?

Вопрос 17.2

Почему внутри обработчика выбора всё равно нужна проверка `if not selection:`?

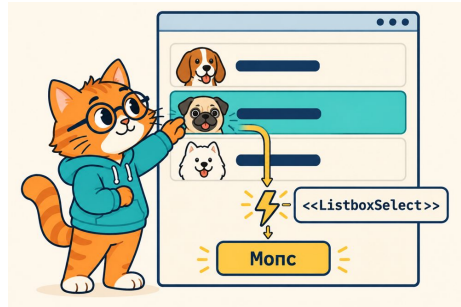
Практика: задача 17 Подпись по выбору

Тренируем: `bind` и событие `<<ListboxSelect>>`, параметр `event`, снятый выбор

Откройте задачу **№17** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: весь файл окна — создание окна, обработчики, `mainloop()`. Каркас — в примечаниях к задаче.

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.

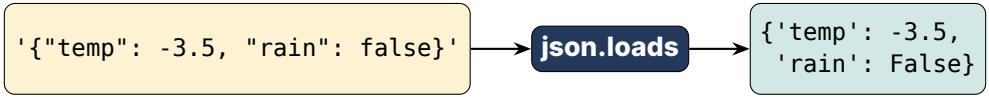


Блок 8

JSON

Задачи 18–20: текст JSON превращается в словари и списки

18 JSON — это пока только текст



В JSON	В Python	В JSON	В Python
`{...}` объект	`dict`	`true` / `false`	`True` / `False`
`[...]` массив	`list`	`null`	`None`
`"текст"`	`str`	`12`, `-3.5`	`int`, `float`

Сервер присылает *текст*. Искать в нём поле срезом ненадёжно: порядок полей и пробелы бывают разными. `json.loads` (модуль `json` есть в Python) разбирает текст целиком.

18 json.loads на примере погоды

```
import json
text = '{"temp": -3.5, "rain": false, "note": null, "tags": ["ветер"]}'
data = json.loads(text)
print(type(text), type(data))
print(data["temp"] + 1, data["rain"], data["note"], data["tags"])
print(json.loads('"a\\/b \\u0414"'))  # \/ → /, \u0414 → Д
```

Вывод

```
<class 'str'> <class 'dict'>
-2.5 False None ['ветер']
a/b Д
```

В JSON слеш иногда пишут как `\/`, а буквы — кодами вроде `\u0414`. `json.loads` раскодирует их сам.

18 Проверьте себя

Вопрос 18.1

`text = '{"n": 5}'`. Что напечатают `print(type(text))` и `print(json.loads(text)["n"] * 2)`?

Вопрос 18.2

Почему `text["n"]` (без `json.loads`) не работает?

Практика: задача 18 Первый JSON

Тренируем: `json.loads`: текст ответа → словарь

Откройте задачу **№18** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.

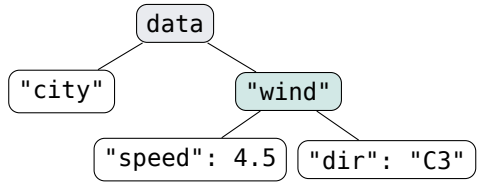


19 Словарь внутри словаря

```
data = {"city": "Сочи", "wind": {"speed": 4.5, "dir": "С3"}}
wind = data["wind"]  # вложенный словарь
print(wind)
print(wind["speed"])
print(data["wind"]["dir"])  # то же в одну строку
print(f"Ветер {data['wind']['speed']} м/с")
```

Вывод

```
{'speed': 4.5, 'dir': 'С3'}
4.5
С3
Ветер 4.5 м/с
```



Каждые квадратные скобки — шаг на уровень ниже: `data["wind"]["dir"]`.

19 Проверьте себя

Вопрос 19.1

Что получится: `data["speed"]` и `data["wind"]["city"]` для словаря с прошлого слайда?

Вопрос 19.2

Что напечатает `print(f"{data['wind']['speed']}")` — `4.5` или `4.50`?

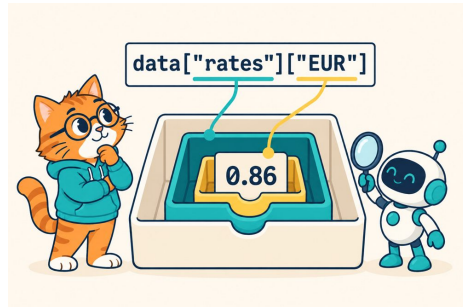
Практика: задача 19 Курс внутри ответа

Тренируем: вложенный словарь: скобки подряд

Откройте задачу **№19** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



20 Ввод в несколько строк: sys.stdin.read()

`input()` читает **одну** строку. Если JSON записан «красиво», на нескольких строках, `input()` вернёт только `{`, и `json.loads` упадёт.

```
import sys
text = sys.stdin.read()  # весь ввод
```

`sys.stdin.read()` читает всё до конца ввода — вместе с переводами строк. Модуль `sys` есть в Python.

Конец ввода в терминале

В Контесте ввод заканчивается сам. Дома после вставки строк нажмите:
Ctrl+D — macOS и Linux;
Ctrl+Z, потом **Enter** — Windows.
Без этого программа ждёт ещё строк. Встроенная консоль редактора может работать иначе — используйте терминал.

20 Пример: маршрут автобуса

```
import json
import sys

text = sys.stdin.read()  # весь ввод
data = json.loads(text)
print("Маршрут", data["route"])
stops = data["stops"]  # массив → список
print("Первая:", stops[0])
```

Ввод

```
{
  "route": "7",
  "stops": ["Вокзал", "Парк",
            "Музей"]
}
```

Вывод

```
Маршрут 7
Первая: Вокзал
```

Ввод на пяти строках разобран целиком. Массив JSON стал обычным списком: элементы — по номеру.

20 len, join и пустой список

```
cities = ["Тверь", "Клин"]
print(len(cities), ", ".join(cities))
print(len([]), "[" + ", ".join([]) + "]")
print(", ".join([]))  # пустая строка вывода
print("конец")
```

Вывод

```
2 Тверь, Клин
0 []

конец
```

`len(список)` — число элементов,
`", ".join(список)` — элементы через разделитель.
Из пустого списка `join` делает пустую строку, и `print` выведет **пустую строку** — это тоже строка вывода.

20 Проверьте себя

Вопрос 20.1

`names = []`. Сколько строк выведут `print(len(names))` и затем `print(", ".join(names))`?

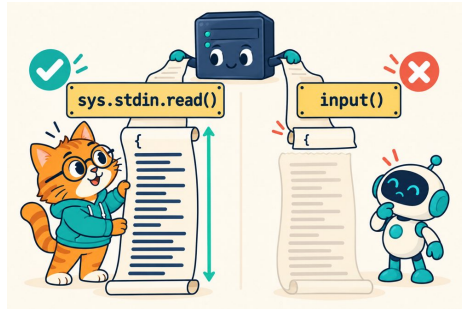
Практика: задача 20 Ответ в несколько строк

Тренируем: `sys.stdin.read()`, список из JSON, `len` и `join`

Откройте задачу **№20** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



Блок 9

JSON: пропуски и поломки

Задачи 21–23: поля, которых нет, текст, который не JSON, и многострочное поле

21 Необязательное поле: get с запасным значением

```
profile = {"name": "Оля"}
print(profile.get("phone"))
print(profile.get("pets", []))
print(profile)  # не изменился
pets = profile.get("pets", [])
if pets:
    print("есть питомцы")
else:
    print("питомцев нет")
```

Вывод

```
None
[]
{'name': 'Оля'}
питомцев нет
```

`d.get(ключ, запас)` возвращает значение, а если ключа нет — запас, без ошибки; без запаса — `None`. Словарь при этом не меняется. Пустой список в `if` — ложь.

21 Три способа прочитать поле

Запись	Ключ есть	Ключа нет
`d["pets"]`	значение	`KeyError`
`d.get("pets")`	значение	`None`
`d.get("pets", [])`	значение	`[]`

Вспомним из задачи 20: массив JSON после `json.loads` — список, `", ".join(список)` склеивает его элементы.

Вопрос 21.1

`profile = {"name": "Оля", "pets": []}`. Что вернут `profile.get("name", "Гость")`, `profile.get("age", 0)` и `bool(profile.get("pets", ["кот"]))`?

Практика: задача 21 Поле забыли прислать

Тренируем: `get` с запасным значением; поля нет и пустой список

Откройте задачу **№21** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



22 Не всякий ответ — JSON

Сервер может прислать страницу HTML, половину ответа или пустоту. Тогда `json.loads` выбрасывает `json.JSONDecodeError` — ловим, как `ValueError` в задаче 06.

```
import json

for text in ['{"a": 1, "b": [2]}', "<html>",
             '{"a": 1', ""]:
    try:
        data = json.loads(text)
        print("JSON:", list(data))
    except json.JSONDecodeError:
        print("не JSON:", repr(text))
```

Вывод

```
JSON: ['a', 'b']
не JSON: '<html>'
не JSON: '{"a": 1'
не JSON: ''
```

Тип ошибки пишется через модуль: `json.JSONDecodeError`. Пустой текст — тоже не JSON. `list(data)` — список ключей.

22 Поля верхнего уровня и ввод целиком

`len(словарь)` — сколько пар **на верхнем уровне**. Вложенные словари не раскрываются:

```
d = {"a": 1, "b": {"c": 2, "d": 3}}
print(len(d))  # 2
```

Ответ может занимать несколько строк: читайте весь ввод через `sys.stdin.read()` (задача 20). В терминале закончите ввод: Ctrl+D, в Windows — Ctrl+Z и Enter.

Вопрос 22.1

Что напечатает `print(len(json.loads('{"x": [1, 2, 3], "y": {}}')))`? А если в тексте одинарные кавычки: `"{'x': 1}"`?

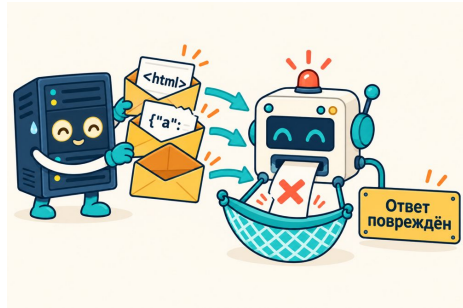
Практика: задача 22 Битый JSON

Тренируем: `try / except` `json.JSONDecodeError`, весь ввод, число полей

Откройте задачу **№22** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



23 Многострочное поле Text

```
note = Text(height=6, width=40)
note.pack()
content = note.get("1.0", "end-1c")
```

Фрагмент: окно уже создано. `Entry` — одна строка, `Text` — много строк.

- Адрес в Text — «строка.символ»: `"1.0"` — первая строка, символ 0.
- `"end"` — конец текста. Text сам хранит в конце служебный `\n`; `"end-1c"` — конец без этого символа.

Пользователь вписал две строки: «Купить чай», Enter, «и печенье».

```
note.get("1.0", "end")      'Купить чай\nи печенье\n'
note.get("1.0", "end-1c")   'Купить чай\nи печенье'
```

23 Пример: заголовок заметки

```
def show_title():
    content = note.get("1.0", "end-1c")  # без служебного \n
    if content.strip() == "":
        result.config(text="Заметка пустая")
        return
    first_line = content.split("\n")[0]
    result.config(text=f"Заголовок: {first_line}")
```

Действие (эмулятор окна)	Надпись
«Заголовок» при пустом поле	Заметка пустая
вписали две строки, «Заголовок»	Заголовок: Купить чай

`content.split("\n")` — список строк текста. Полный файл: `demos/g23_note.py`.

23 Проверьте себя

Вопрос 23.1

В Text ввели одну строку `abc` без Enter. Что вернут `note.get("1.0", "end")` и `note.get("1.0", "end-1c")`?

Вопрос 23.2

`data = {"code": 404, "status": "error"}` и `name = "code"`. Что даст `f"{name} = {data[name]}"`?

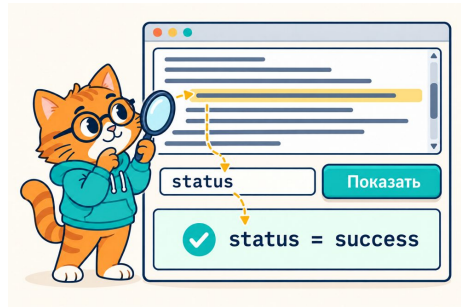
Практика: задача 23 JSON-просмотрщик

Тренируем: `Text: get("1.0", "end-1c")`, JSON из поля, проверки по порядку

Откройте задачу **№23** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: весь файл окна — создание окна, обработчики, `mainloop()`. Каркас — в примечаниях к задаче.

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



Блок 10

Ответ API

Задачи 24–27: статус ответа, точный формат, адрес запроса и первый настоящий запрос

24 Сначала — что случилось, потом данные

Сервис доставки (ответы уже разобраны в словари):

```
answers = [{"state": "ok", "data": {"town": "Тверь"}},
           {"state": "fail", "reason": "Нет трек-номера"}]
for reply in answers:
    if reply["state"] == "ok":  # сначала — что случилось
        print("Доставка:", reply["data"]["town"])
    else:
        print("Проблема:", reply["reason"])
```

Вывод

```
Доставка: Тверь
Проблема: Нет трек-номера
```

При успехе данные лежат в одном поле, при ошибке — описание проблемы в другом. Пока не проверено поле состояния, неизвестно, что лежит в ответе.

24 Части адреса: split(”/”)

```
url = "https://shop.example/catalog/tea/green.html"
parts = url.split("/")
print(parts)
print(parts[3], parts[4])
```

Вывод

```
['https:', '', 'shop.example', 'catalog', 'tea', 'green.html']
catalog tea
```

`split("/")` режет по каждому `/`; два `/` подряд дают пустую часть `''`. Номера частей — с нуля.

24 Проверьте себя

Вопрос 24.1

Что вернут `"https://a.example/x/y/z.png".split("/")[4]` и `"Нет".split("/")[4]`?

Вопрос 24.2

Ответ сервиса доставки — `{"state": "fail", "reason": "Нет трек-номера"}`. Ученик сразу режет `reason` по «/» и берёт часть [3]. Что случится?

Практика: задача 24 Сервер ответил

Тренируем: сначала `status`, потом данные; части адреса

Откройте задачу **№24** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



25 Ровно два знака после точки

```
price = 94.4767
print(f"{price:.2f}")  # 94.48
print(f"{2.5:.2f}")  # 2.50 — оба знака всегда
print(round(2.5, 2))  # 2.5 — round нули не дописывает
print("ok" == "OK", "ok" == "ok ")  # сравнение точное
```

Вывод

```
94.48
2.50
2.5
False False
```

`f"{x:.2f}"` округляет до двух знаков и **всегда** печатает оба, дописывая нули.
`round(x, 2)` только округляет число: нули он не дописывает.
Строки сравниваются точно: регистр и пробелы важны.

25 Порядок: сначала результат, потом курс

Успех

```
{"result": "success",
 "base_code": "USD",
 "rates": {"RUB": 81.25}}
```

Ошибка

```
{"result": "error",
 "error-type": "unsupported-code"}
```

В ответе с ошибкой нет ни `rates`, ни `base_code`: обращение к ним даст `KeyError`. Поэтому сначала `result`.

Нужного кода может не быть и в `rates` успешного ответа. Ключ во вложенном словаре проверяют так же: `"GBP" in data["rates"]`.

Вопрос 25.1

Что напечатают `print(f"{0.1 + 0.2:.2f}")` и `print("done" == "Done")`?

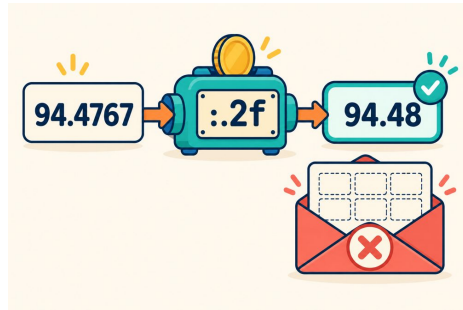
Практика: задача 25 Курс из ответа

Тренируем: проверка `result`, наличие ключа, формат `:.2f`

Откройте задачу **№25** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



26 split() без аргументов и адрес из частей

```
text = "  Nizhny   NOVGOROD "
print(text.split(" "))  # пустые кусочки остаются
words = text.lower().split()  # по любым пробелам, без пустых
print(words, len(words))
city = "-".join(words)
print(f"https://weather.example/api/{city}/today")
```

Вывод

```
['', '', 'Nizhny', '', '', 'NOVGOROD', '']
['nizhny', 'novgorod'] 2
https://weather.example/api/nizhny-novgorod/today
```

`split(" ")` оставляет пустые кусочки, `split()` без аргументов — нет. `len(words)` — число слов. Адрес собираем f-строкой.

26 Проверьте себя

Вопрос 26.1

Что вернут `"  Hound  ".lower().split()` и `len("a  b c".split())`?

Вопрос 26.2

`words = ["nizhny", "novgorod"]`. Как получить `nizhny/novgorod`, не зная заранее число слов? А если слов точно два?

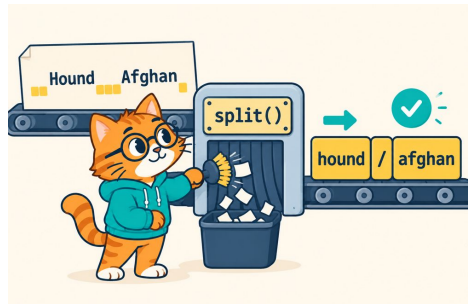
Практика: задача 26 Адрес запроса породы

Тренируем: `split()` без аргументов, число слов, адрес из частей

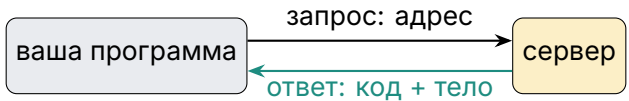
Откройте задачу **№26** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



27 Запрос к серверу



requests — библиотека для запросов. Её ставят один раз в окружение, которым запускаете программы:

```
python -m pip install requests
```

(или `python3 …`). Модули `json` и `sys` уже есть в Python.

В Контесте ничего ставить не нужно: вместо интернета отвечает подготовленный тест.

Для показа без интернета у нас учебный сервер погоды: `python demo_server.py` в отдельном терминале. Он отвечает на `http://127.0.0.1:8765/weather/…`

27 requests.get и объект ответа

```python
import requests

url = "http://127.0.0.1:8765/weather/kazan"
response = requests.get(url)  # здесь идёт запрос
print(response.status_code)  # HTTP-код
print(response.text[:26])  # тело — это текст
data = response.json()  # тело → словарь
print(data["city"], data["temp"])
```

Вывод

```
200
{"city": "Казань", "temp":
Казань -3.5
```

`requests.get(url)` — здесь программа идёт по сети и ждёт ответа. `response` — объект ответа: HTTP-код `status_code`, тело-текст `text` и метод `json()`. Адрес — учебный сервер `demo_server.py`.

27 json.loads(text) и response.json()

	Откуда текст	Что получаем
`json.loads(text)`	строка, которая уже есть в программе	словарь
`response.json()`	тело ответа сервера	словарь

`response.json()` делает то же, что `json.loads(response.text)`, только короче. Для ответа сервера отдельный `json.loads` не нужен.

27 Запрос — в обработчике, а не при запуске

✔ **Так**: запрос внутри функции кнопки

```
def show_weather():
    response = requests.get(URL)  # запрос — только при нажатии
    data = response.json()
    result.config(text=f"{data['city']}: {data['temp']} °C")
```

✖ **Не так** (фрагмент)

```
response = requests.get(URL)  # выполнится при запуске!

def show_weather():
    data = response.json()  # каждый раз старый ответ
```

Запрос вне функции выполнится один раз — при запуске; кнопка покажет старый ответ. Эмулятор: два нажатия → два запроса.

27 Проверьте себя

Вопрос 27.1

Пользователь нажал кнопку три раза. Сколько запросов отправит правильная программа? А программа «Не так»?

Вопрос 27.2

Чем `response.text` отличается от `response.json()`?

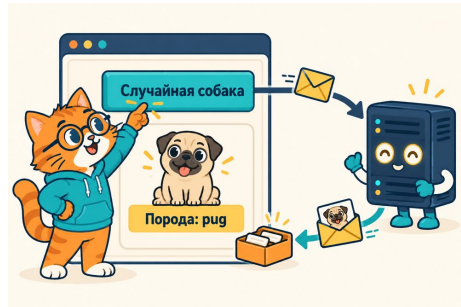
Практика: задача 27 Случайная собака

Тренируем: `requests.get` и `response.json()` в обработчике кнопки. Фото в окне не нужно — только порода

Откройте задачу **№27** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: весь файл окна — создание окна, обработчики, `mainloop()`. Каркас — в примечаниях к задаче.

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



Блок 11

HTTP и ошибки сервера

Задачи 28–30: коды ответа, порядок проверок и ошибки в окне

28 Код ответа HTTP

2xx — успех	4xx — ошибка в запросе	5xx — сломался сервер
200	404 «не найдено»	500

- Кроме тела сервер присылает трёхзначный код: `response.status_code`. По первой цифре видно, чем кончилось дело.
- Тело может выглядеть прилично даже при ошибке: аккуратный JSON с кодом 404.
- `response.raise_for_status()` молчит при кодах до 399 и выбрасывает `requests.HTTPError` при кодах 400–599.

28 Пример на учебном сервере

```python
import requests

BASE = "http://127.0.0.1:8765/weather/"
for place in ["kazan", "atlantis", "broken"]:
    response = requests.get(BASE + place)
    print(place, response.status_code)
    try:
        response.raise_for_status()
        print("  тишина")
    except requests.HTTPError:
        print("  HTTPError")
```

Вывод

```
kazan 200
  тишина
atlantis 404
  HTTPError
broken 500
  HTTPError
```

Для `atlantis` сервер ответил кодом 404 и JSON-телом с ошибкой, для `broken` — кодом 500 и страницей HTML. `raise_for_status()` смотрит только на код, а не на тело.

28 Код HTTP и поле status — разные уровни

	HTTP-код	Поле `status` / `result` в JSON
Кто сообщает	протокол HTTP	сам сервис, внутри тела
Где лежит	`response.status_code`	`data["status"]` после разбора
Пример	404, 500	`"error"`

Бывает код 200, а в JSON — `"status": "error"`: связь и HTTP в порядке, но сервис не смог выполнить просьбу.

Вопрос 28.1

Цепочка: сначала `if code >= 400:`, потом `elif code == 404:`.
Сработает ли ветка для 404? Как это исправить?

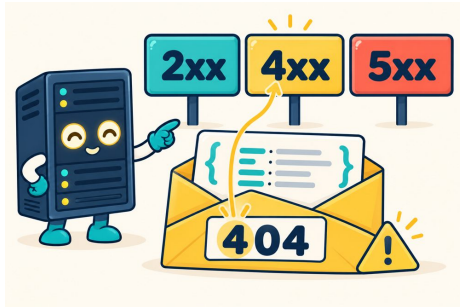
Практика: задача 28 Код ответа

Тренируем: классы кодов HTTP, когда `raise_for_status` выбрасывает исключение

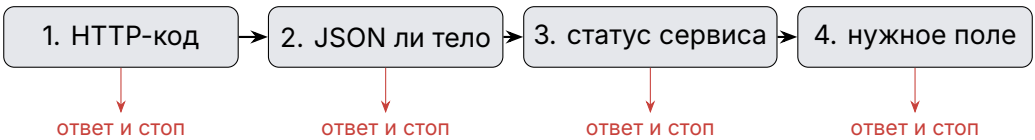
Откройте задачу **№28** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



29 Уровни проверки ответа



- Каждая проверка опирается на предыдущую: поля читают только после разбора JSON, статус — только из словаря.
- Порядок ошибок задаёт заказчик: при коде 500 и испорченном теле сообщаем именно об HTTP-ошибке.
- Первая неудачная проверка даёт ответ, дальше не идём.
- Поля может не быть совсем: `data.get("status") != "success"` верно и для отсутствующего поля, и для другого значения.

29 Функция с return на каждом шаге

```python
def check_parcel(weight, address):
    if weight > 30:
        return "Слишком тяжело"
    if address == "":
        return "Нет адреса"
    return "Принято"


print(check_parcel(40, ""))
print(check_parcel(5, ""))
print(check_parcel(5, "Тверь"))
```

Вывод

```
Слишком тяжело
Нет адреса
Принято
```

return сразу заканчивает функцию. Проверки идут сверху вниз; после первой сработавшей остальные не выполняются, длинная цепочка **elif** не нужна.

29 Первая строка отдельно, остальное целиком

```python
import sys

count = int(input())  # первая строка
rest = sys.stdin.read()  # остальное
print("число:", count)
print("остальное:", repr(rest))
```

Ввод

```
3
первая строка
вторая строка
```

Вывод

```
число: 3
остальное: 'первая строка\nвторая строка\n'
```

`input()` забирает первую строку, `sys.stdin.read()` — всё, что осталось. Их можно использовать вместе.

29 Проверьте себя

Вопрос 29.1

Что напечатает `print(check_parcel(40, "Тверь"))` для функции с прошлого слайда?

Вопрос 29.2

Код ответа 204, тело — пустая строка. На каком уровне из четырёх программа остановится?

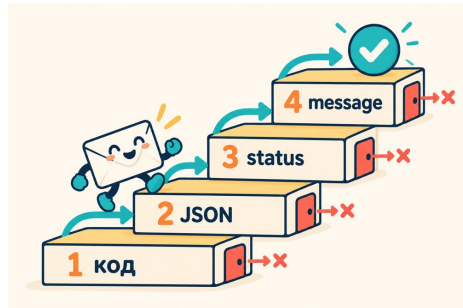
Практика: задача 29 Четыре проверки по очереди

Тренируем: порядок проверок ответа: код → JSON → статус → поле

Откройте задачу **№29** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: консольную программу. Печатайте только ответ, без «Введите...».

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



30 Выпадающий список ttk.Combobox

```
from tkinter import ttk
box = ttk.Combobox(values=["kazan", "sochi"],
                   state="readonly")
box.pack()
place = box.get()  # пока не выбрали — ""
```

Фрагмент: окно уже создано.

- Значение не печатают, а выбирают из списка `values`.
- `ttk` подключается отдельно, как `messagebox`.
- `state="readonly"` запрещает вписывать текст руками.
- `get()` — выбранное значение; ничего не выбрано — пустая строка.

30 Что может пойти не так с запросом

Что случилось	Как проявляется	Как поймать
Связи нет	`requests.ConnectionError` в `get`	`except requests.RequestException`
Сервер долго молчит	`requests.Timeout`, если задан `timeout`	тот же `except`
Код 400–599	`HTTPError` после `raise_for_status()`	тот же `except`
Тело не JSON	ошибка разбора в `response.json()`	тот же `except`, если `json()` в `try`
Сервис сообщил об ошибке	код 200, но поле состояния не `"success"`	обычный `if`

`RequestException` — «семья» ошибок запроса: `ConnectionError`, `Timeout`, `HTTPError` и ошибка разбора в `response.json()`.

30 Ловим ошибки запроса

```
import requests
BASE = "http://127.0.0.1:"
for url in [BASE + "8765/weather/broken",
            BASE + "8765/weather/slow",
            BASE + "8799/weather/kazan"]:
    try:
        response = requests.get(url, timeout=2)
        response.raise_for_status()
    except requests.HTTPError:
        print("HTTPError", response.status_code)
    except requests.RequestException:
        print("нет ответа")
```

Вывод

```
HTTPError 500
нет ответа
нет ответа
```

`except` — сверху вниз, срабатывает первый подходящий. `timeout=2`: сервер молчит дольше 2 с — `Timeout`.

30 Окно погоды: запрос внутри try

```
def show():
    try:
        response = requests.get(BASE + box.get(), timeout=5)
        response.raise_for_status()  # 400–599 → HTTPError
    except requests.RequestException:  # связь, таймаут, HTTP
        result.config(text="Сервер не ответил")
        return
    data = response.json()
    if data["status"] == "success":
        result.config(text=f"{data['city']}: {data['temp']} °C")
    else:
        result.config(text=data["message"])  # ошибка сервиса
```

Пример выводит всё в надпись — окна ошибок `showerror` вы знаете по задаче 10. После `except` и `return` код ниже выполняется, только если запрос удался.

30 Окно погоды: что видно после нажатия

Город	Ответ учебного сервера	Надпись (эмулятор окна)
kazan	200, `status: success`	Казань: -3.5 °C
sochi	200, `status: success`	Сочи: 12.4 °C
mars	200, `status: error`	Нет данных для этого места
broken	500, страница HTML	Сервер не ответил

После каждой ошибки окно работает дальше: новый выбор и нажатие снова делают запрос. В примере сразу выбран первый город (`box.set("kazan")`); если ничего не выбрано, `box.get()` вернёт пустую строку. Полный файл: `demos/g30_weather.py`.

30 Проверьте себя

Вопрос 30.1

В окне погоды забыли `raise_for_status()`. Сервер ответил кодом 500 и страницей HTML. Что случится в строке `data = response.json()` после `try`?

Вопрос 30.2

Сервер ответил кодом 200 и `{"status": "error", "message": "Нет данных"}`. Сработает ли `except requests.RequestException`?

Практика: задача 30 Валюты

Тренируем: `ttk.Combobox`, ошибки запроса в окне: `RequestException`, `raise_for_status`, `showerror`

Откройте задачу **№30** в нашем Контесте и прочитайте условие целиком: ввод, вывод, примечания.

Отправляете: весь файл окна — создание окна, обработчики, `mainloop()`. Каркас — в примечаниях к задаче.

Компилятор — **(make) Python**. Сначала проверьте программу у себя на примерах.



Что мы прошли

01–10. Точный вывод, границы, порядок проверок, индексы и срезы, `try / except`, обработчик кнопки, время в минутах, окно ошибки.

11–20. `lower`, `endswith`, Listbox и видимый список, словарь, событие выбора, JSON, вложенные словари, весь ввод.

21–30. `get` с запасом, повреждённый JSON, многострочное поле, статус ответа, `:.2f`, адрес из частей, `requests`, коды HTTP, порядок проверок, ошибки запроса в окне.

Дальше. Продолжайте с первой нерешённой задачи. Не получается — сообщение чекера, подсказки в примечаниях, вопрос преподавателю.